Akademia Górniczo-Hutnicza
w Krakowie

Faculty: AGH	Academic Year 2025/2026	Year of study	Fields of science:
Student(s) name:	Simon GANIER-LOMBARD & Daniil STEPANOV		
Course:	**Real Time Operating Systems in Control Applications**		
Exercise nr:	**RTOS-06**		
Exercise subject:	**Processes priorities and scheduling algorithms**		
Lecturer:	dr inż. Grzegorz Hayduk		
Date: 02.06.2026			Grade:

1 Exercise purpose

The goal of this exercise is to learn how priorities and scheduling policies work in QNX.
We create child processes with fork, and after give them different priorities, scheduling policies, and make each one run a heavy computation with no blocking calls.
Then we then look at the order in which they finish to see the effect of the priority and of the policy on this processes.

2 Assumptions / Theory

In QNX every process has a priority from 1 to 255, where 255 is the highest priority. The scheduler always runs the ready process with the highest priority, and a higher priority process can preempt a lower one.
There are three scheduling policies type:
- SCHED_FIFO: the process runs until the CPU it's by itself or a higher priority process preempts it.
- SCHED_RR: same as FIFO, but the process also gives up the CPU when it uses its time quantum.
- SCHED_SPORADIC: the priority drops by 1 when the budget runs out and is restored after the process blocks.

We set the priority with *sched_setparam* and the policy with *sched_setscheduler*, and we read them back with *sched_getparam* and *sched_getscheduler.*

The fork call makes a child process that is a copy of the parent; it returns 0 in the child and the child pid in the parent.

3 Description of implementation

We wrote two programs.
Both use fork to create children, shared memory for the results, and a semaphore so all children start at the same moment.

- ex6_1.c covers points 1 and 2.

It creates 10 children with priorities from 10 to 100. The parent opens two shared memory areas: one for the results and one for a semaphore gate.
Each child maps the same memory, sets its priority, prints READY, then waits on the semaphore.

The parent waits 2 seconds, posts the semaphore 10 times to release all children together, and waits for them to finish.
Each child measures its own run time with *gettimeofday* and writes it to shared memory. The parent prints the time for every priority at the end.

- ex6_2.c covers points 3 and 4.
It creates 9 children, 3 for each policy (SCHED_FIFO, SCHED_RR, SCHED_SPORADIC), all with the same priority 20.
It uses two semaphores: a ready semaphore so the parent knows every child is set up, and a gate semaphore to release them all at once.
Each child records its start rank and finish rank with an atomic counter, so we can see the order in which they start and finish.
Each child also reads back its real policy with *sched_getscheduler.*
The computation in both programs is a plain loop with no blocking calls, so the only thing that changes the finish order is the priority or the policy.

See the full commented code in the .zip file.

4 Conclusions

This exercise showed us how the QNX scheduler uses priority to choose which process runs. In the first program the children with higher priority finished before the ones with lower priority, which matches the preemptive rule. In the second program the policy changed the finish order even though all children had the same priority.
The tricky part was to make all children start at the same time for the comparison to be fair. We solved this with a semaphore gate: every child waits on the semaphore and the parent releases them together. We also used shared memory, so, so the children can write their results back to the parent. One point to remember is to always check the return value of the sched_ functions, because if they return -1 the priority or policy did not change.
We saw that SCHED_SPORADIC may not be fully active on the lab QNX version, so its behaviour can look like another policy.